

Semana 4 - Redes Neurais Profundas

João Florindo

Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas - Brasil
florindo@unicamp.br

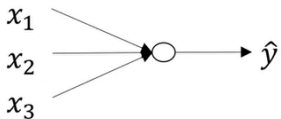
Outline

- 1 Redes Neurais Profundas
- 2 Forward e Backward Propagation
- 3 Representações Profundas
- 4 Blocos da Rede Neural
- 5 Hiperparâmetros

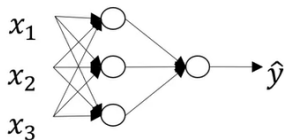
- Regressão logística pode ser interpretada como uma “rede neural de uma camada”
- Vimos também a rede de uma camada escondida (rede de 2 camadas)

- Regressão logística pode ser interpretada como uma “rede neural de uma camada”
- Vimos também a rede de uma camada escondida (rede de 2 camadas)

- Regressão logística pode ser interpretada como uma “rede neural de uma camada”
- Vimos também a rede de uma camada escondida (rede de 2 camadas)

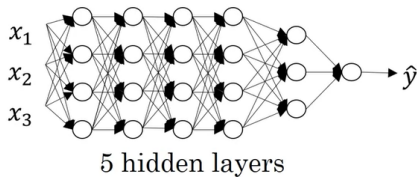
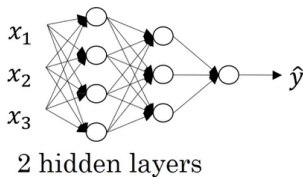


logistic regression



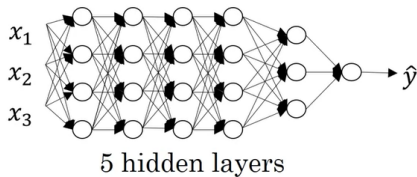
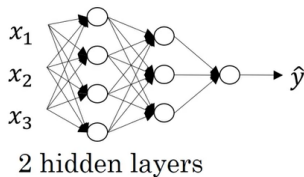
1 hidden layer

- Podemos naturalmente aumentar o número de camadas



- A regressão logística é o mais “raso” entre esses modelos e a rede de 5 camadas escondidas é o mais “profundo”

- Podemos naturalmente aumentar o número de camadas



- A regressão logística é o mais “raso” entre esses modelos e a rede de 5 camadas escondidas é o mais “profundo”

Notação

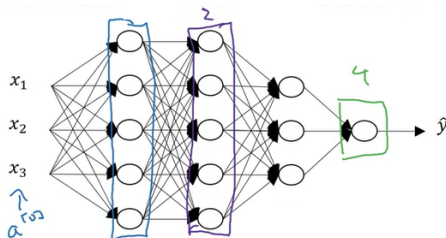
Notação	Definição	Dimensão
L	Número de camadas	
$n^{[0]}$	número de neurônios na camada de entrada	
$n^{[l]}$	número de neurônios na l -ésima camada, l de 1 a L	
$W^{[l]}$	pesos da l -ésima camada	$(n^{[l]}, n^{[l-1]})$
$b^{[l]}$	<i>bias</i> da l -ésima camada	$(n^{[l]}, 1)$
$Z^{[l]}$	soma linear da l -ésima camada	$(n^{[l]}, m), Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}$
$g^{[l]}$	função de ativação da l -ésima camada	
$A^{[l]}$	ativação da l -ésima camada	$(n^{[l]}, m), A^{[l]} = g^{[l]}(Z^{[l]})$

Outline

- 1 Redes Neurais Profundas
- 2 Forward e Backward Propagation**
- 3 Representações Profundas
- 4 Blocos da Rede Neural
- 5 Hiperparâmetros

Forward

Repetir o cálculo para mais camadas:



$$z^{[1]} = W^{[1]}a^{[0]} + b^{[1]}$$

$$a^{[1]} = g^{[1]}(z^{[1]})$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[2]} = g^{[2]}(z^{[2]})$$

$$\vdots$$

$$z^{[4]} = W^{[4]}a^{[3]} + b^{[4]}$$

$$a^{[4]} = g^{[4]}(z^{[4]})$$

Em geral:

$$\begin{cases} z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]} \\ a^{[l]} = g^{[l]}(z^{[l]}) \end{cases}$$

Código

O *forward propagation*, já vetorizado, de modo que $A^{[0]} = X$ se resume a

$$Z[l] = W[l] * A[l-1] + b[l]$$

$$A[l] = \text{sigmoid}(Z[l])$$

NOTA: Para cada uma das L camadas ainda temos um *loop for*; isso não temos como vetorizar

Código - 2 Camadas

O *backward* é feito de trás para frente usando a regra da cadeia:

```
# backpropagation
dZ[2] = A[2] - Y # obtido pela combinacao da derivada
# da funcao de custo com g'[2]
dW[2] = 1/m * np.matmul(dZ[2], A[1].T)
db[2] = 1/m * np.sum(dZ[2], axis=1, keepdims=True)
dZ[1] = np.multiply(np.matmul(W[2].T, dZ[2]), g'[1](Z[1]))
# derivada da ativacao eh usada aqui
dW[1] = 1/m * np.matmul(dZ[1], X.T)
db[1] = 1/m * np.sum(dZ[1], axis=1, keepdims=True)

# atualizacao de parametros
W[1] = W[1] - learning_rate * dW[1]
b[1] = b[1] - learning_rate * db[1]
W[2] = W[2] - learning_rate * dW[2]
b[2] = b[2] - learning_rate * db[2]
```

Código - L Camadas

```
# backpropagation
for l in range(L):
    dZ[l] = dA[l] * g[l]'(Z[l])
    dW[l] = 1/m * np.matmul(dZ[l], A[l-1].T)
    db[l] = 1/m * np.sum(dZ[l], axis=1, keepdims=True)
    dA[l-1] = np.matmul(W[l].T, dZ[l])

# atualizacao de parametros
W[l] = W[l] - learning_rate * dW[l]
b[l] = b[l] - learning_rate * db[l]
```

Checagem das dimensões

- Uma forma simples de se debugar um código é checando as dimensões das matrizes e vetores envolvidos:

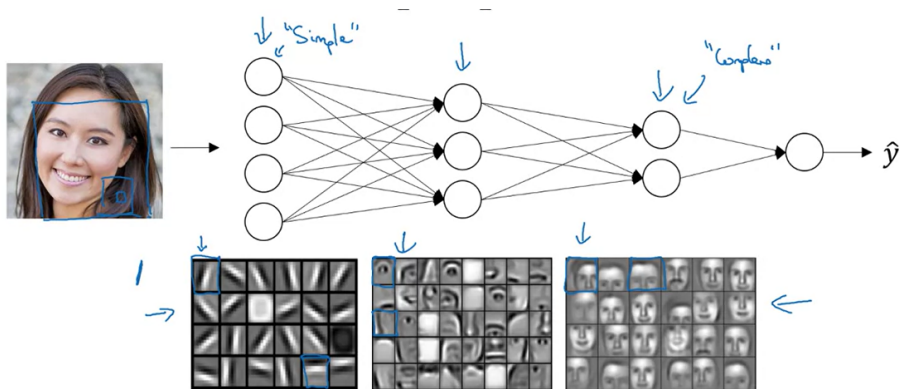
$W^{[l]}$	$(n^{[l]}, n^{[l-1]})$
$b^{[l]}$	$(n^{[l]}, 1)$
$Z^{[l]}$	$(n^{[l]}, m)$
$A^{[l]}$	$(n^{[l]}, m)$
$dW^{[l]}$	$(n^{[l]}, n^{[l-1]})$
$db^{[l]}$	$(n^{[l]}, 1)$
$dZ^{[l]}$	$(n^{[l]}, m)$
$dA^{[l]}$	$(n^{[l]}, m)$

Outline

- 1 Redes Neurais Profundas
- 2 Forward e Backward Propagation
- 3 Representações Profundas**
- 4 Blocos da Rede Neural
- 5 Hiperparâmetros

Por que Representações Profundas?

- EXEMPLO: Reconhecimento facial
- 1ª camada detecta *features* de baixo nível (arestas, etc.)
- 2ª camada identifica partes do rosto
- 3ª camada identifica tipos de faces



Por que Representações Profundas?

- Válido para outros tipos de dados. EXEMPLO: Áudio
- *Features* (ruído, sibilância, entonação) → fonemas → palavras → sentenças
- Analogia com o cérebro (questionável!)

Por que Representações Profundas?

- Válido para outros tipos de dados. EXEMPLO: Áudio
- *Features* (ruído, sibilância, entonação) → fonemas → palavras → sentenças
- Analogia com o cérebro (questionável!)

Por que Representações Profundas?

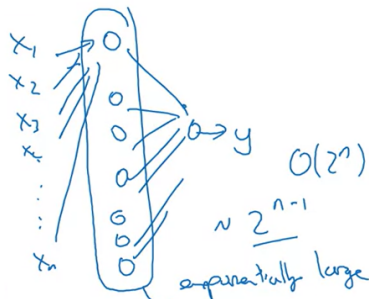
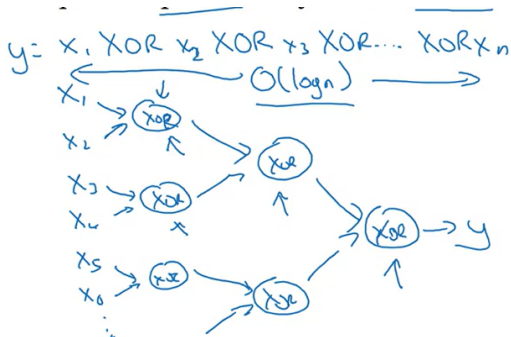
- Válido para outros tipos de dados. EXEMPLO: Áudio
- *Features* (ruído, sibilância, entonação) → fonemas → palavras → sentenças
- Analogia com o cérebro (questionável!)

Teoria de Circuitos

- Outra intuição parte da representação de funções usando portas lógicas
- Suponha a operação XOR entre n entradas

Teoria de Circuitos

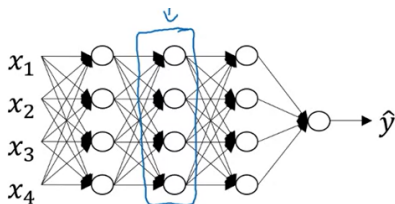
- Outra intuição parte da representação de funções usando portas lógicas
- Suponha a operação XOR entre n entradas



Outline

- 1 Redes Neurais Profundas
- 2 Forward e Backward Propagation
- 3 Representações Profundas
- 4 Blocos da Rede Neural**
- 5 Hiperparâmetros

- Em uma camada específica, calcular a e z (*forward*) assim como as derivadas da , dw , db (*backward*)
- Cache de z no *forward* porque será usado no *backward*



layer l : $W^{[l]}, b^{[l]}$

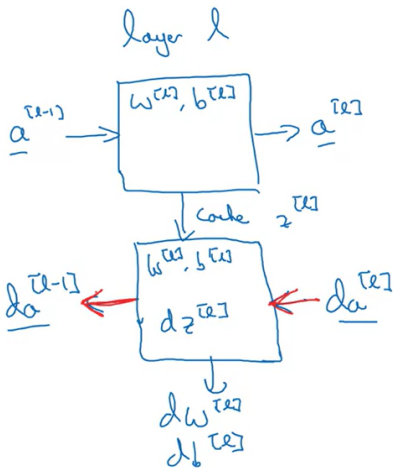
→ Forward: Input $a^{[l-1]}$, output $a^{[l]}$

$$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]} \quad \text{cache } z^{[l]}$$

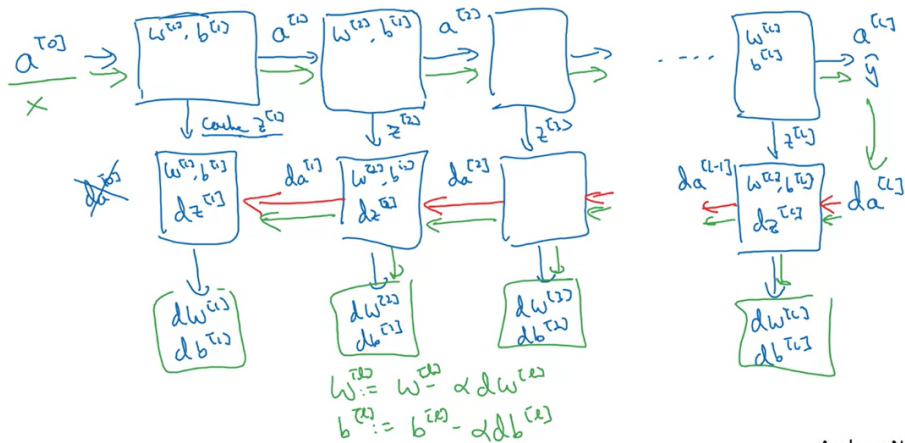
$$\underline{a^{[l]}} = g^{[l]}(z^{[l]})$$

→ Backward: Input $da^{[l]}$, output $da^{[l-1]}$

cache $(z^{[l]})$
 $\frac{dw^{[l]}}{dz^{[l]}}$
 $\frac{db^{[l]}}{dz^{[l]}}$



Uma iteração



NOTA: Para efeitos de implementação, é recomendável que não apenas $z^{[l]}$ seja mantido em cache, $W^{[l]}$ e $b^{[l]}$ também

Forward

- Entrada: $a^{[l-1]}$
- Saídas: $a^{[l]}$ e *cache* $z^{[l]}$

Equações:

$$z^{[l]} = W^{[l]} \cdot a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = g^{[l]}(z^{[l]})$$

Vertigal:

$$z^{[l]} = W^{[l]} \cdot A^{[l-1]} + b^{[l]}$$

$$A^{[l]} = g^{[l]}(z^{[l]})$$

Backward

- Entrada: $da^{[l]}$
- Saídas: $da^{[l-1]}$, $dW^{[l]}$, $db^{[l]}$

Equações:

$$dz^{[l]} = da^{[l]} * g^{[l]'}(z^{[l]})$$

$$dW^{[l]} = dz^{[l]} \cdot \frac{a^{[l-1]T}}{n}$$

$$db^{[l]} = dz^{[l]}$$

$$da^{[l-1]} = W^{[l]T} \cdot dz^{[l]}$$

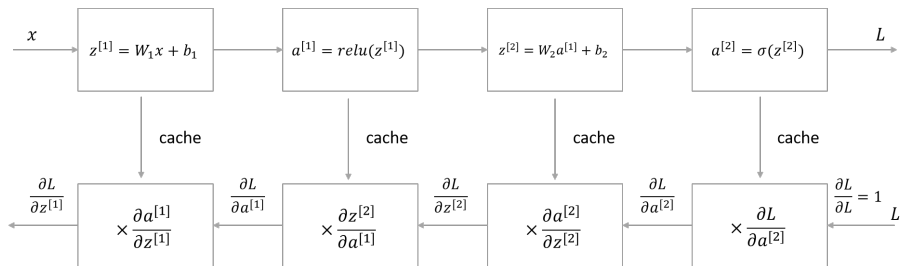
$$dz^{[l]} = \frac{1}{n} \sum da^{[l]} * g^{[l]'}(z^{[l]})$$

$$dW^{[l]} = \frac{1}{n} dz^{[l]} \cdot A^{[l-1]T}$$

$$db^{[l]} = \frac{1}{n} \text{np.sum}(dz^{[l]}, \text{axis}=1, \text{keepdims}=True)$$

$$dA^{[l-1]} = W^{[l]T} \cdot dz^{[l]}$$

Resumo



NOTA: Se a *loss* for a *cross-entropy* binária, temos

$$da^{[L]} = -\frac{y}{a} + \frac{1-y}{1-a}$$

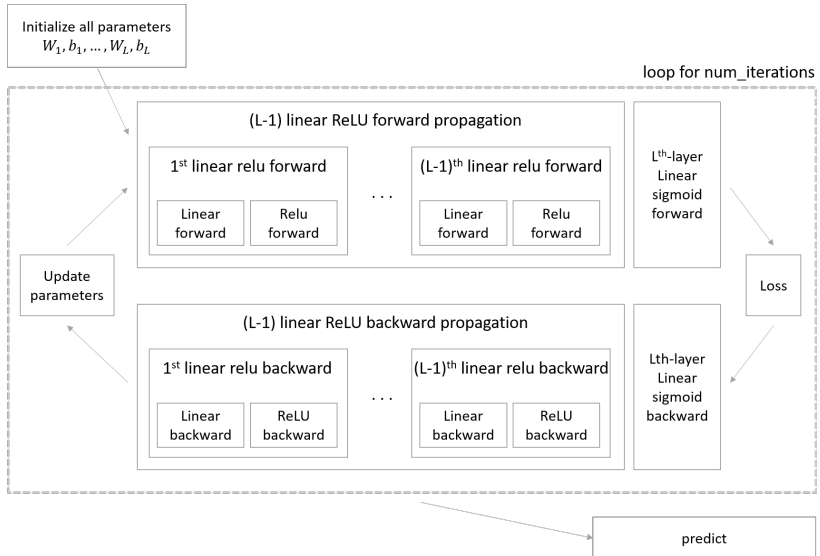
ou ainda vetorizando

$$dA^{[L]} = \left(-\frac{y^{(1)}}{a^{(1)}} + \frac{1-y^{(1)}}{1-a^{(1)}}, \dots, -\frac{y^{(m)}}{a^{(m)}} + \frac{1-y^{(m)}}{1-a^{(m)}} \right)$$

Passos Gerais

- ❶ Inicialize parâmetros / Defina hiperparâmetros
- ❷ Repita para num_{iter} :
 - ❶ *Forward propagation*
 - ❷ Calcule a função de custo
 - ❸ *Backward propagation*
 - ❹ Atualize os parâmetros usando os próprios parâmetros e as derivadas do backprop
- ❸ Use os parâmetros treinados para fazer previsões

Passos Gerais



Outline

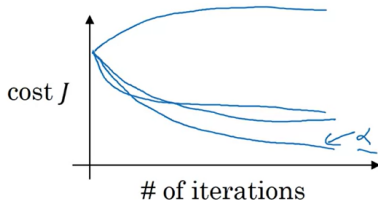
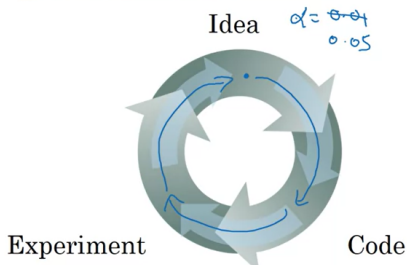
- 1 Redes Neurais Profundas
- 2 Forward e Backward Propagation
- 3 Representações Profundas
- 4 Blocos da Rede Neural
- 5 Hiperparâmetros

- Parâmetros são os W e b aprendidos no treinamento
- Hiperparâmetros controlam o algoritmo de aprendizado em si. EX.:
 - Números de unidades escondidas $n^{[l]}$
 - Taxa de aprendizado α
 - Número de iterações
 - Número de camadas L
 - Tipos de funções de ativação
- E veremos outros mais para frente, como momento, tamanho de *minibatch*, nível de regularização, etc.

- Parâmetros são os W e b aprendidos no treinamento
- Hiperparâmetros controlam o algoritmo de aprendizado em si. EX.:
 - Números de unidades escondidas $n^{[l]}$
 - Taxa de aprendizado α
 - Número de iterações
 - Número de camadas L
 - Tipos de funções de ativação
- E veremos outros mais para frente, como momento, tamanho de *minibatch*, nível de regularização, etc.

- Parâmetros são os W e b aprendidos no treinamento
- Hiperparâmetros controlam o algoritmo de aprendizado em si. EX.:
 - Números de unidades escondidas $n^{[l]}$
 - Taxa de aprendizado α
 - Número de iterações
 - Número de camadas L
 - Tipos de funções de ativação
- E veremos outros mais para frente, como momento, tamanho de *minibatch*, nível de regularização, etc.

- Processo de testar hiperparâmetros é empírico e cíclico



- Intuições de hiperparâmetros para uma determinada aplicação nem sempre são válidas em outra, entre NLP e CV, por exemplo

E o Cérebro?

$$dZ^{[L]} = A^{[L]} - Y$$

$$dW^{[L]} = \frac{1}{m} dZ^{[L]} A^{[L-1]T}$$

$$db^{[L]} = \frac{1}{m} np.sum(dZ^{[L]}, axis = 1, keepdims = True)$$

$$dZ^{[L-1]} = W^{[L]T} dZ^{[L]} * g'^{[L-1]}(Z^{[L-1]})$$

$$\vdots$$

$$dZ^{[1]} = W^{[2]T} dZ^{[2]} * g'^{[1]}(Z^{[1]})$$

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} A^{[0]T}$$

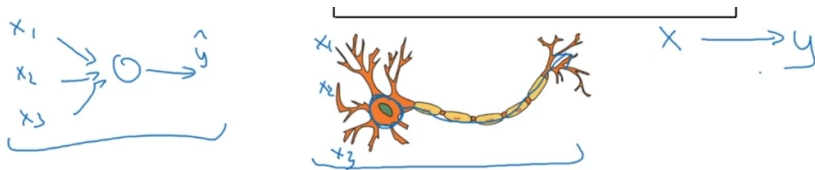
$$db^{[1]} = \frac{1}{m} np.sum(dZ^{[1]}, axis = 1, keepdims = True)$$

E o Cérebro?

- Dizer que todas essas equações fazem algo “igual ao cérebro” é uma simplificação muito exagerada
- Diz-se que uma regressão logística simula muito simplificadaamente um único neurônio
- Pouco se sabe sobre o funcionamento desses neurônios e menos ainda sobre seu “algoritmo” de aprendizado

E o Cérebro?

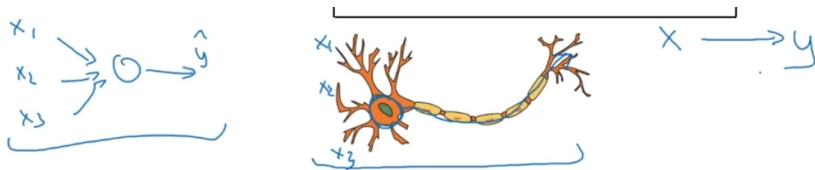
- Dizer que todas essas equações fazem algo “igual ao cérebro” é uma simplificação muito exagerada
- Diz-se que uma regressão logística simula muito simplificado um único neurônio



- Pouco se sabe sobre o funcionamento desses neurônios e menos ainda sobre seu “algoritmo” de aprendizado

E o Cérebro?

- Dizer que todas essas equações fazem algo “igual ao cérebro” é uma simplificação muito exagerada
- Diz-se que uma regressão logística simula muito simplificado um único neurônio



- Pouco se sabe sobre o funcionamento desses neurônios e menos ainda sobre seu “algoritmo” de aprendizado